



國立陽明交通大學
NATIONAL YANG MING CHIAO TUNG UNIVERSITY

AI-Based Cross-Level Attack Detection and Response in Industrial Control Systems

2026/05/15

Student: Quy Pham Dinh

Advisor: Prof. Ying-Dar Lin

High Speed Network Lab

National Yang Ming Chiao Tung University, Taiwan

Outline

- Motivation
- Background
 - Industrial Control Systems (ICS)
 - MITRE ATT&CK Framework for ICS
 - Intrusion Detection System in OT
 - AI Agent
 - Reinforcement Learning
- Related work
- Issues
- Notation
- Problem Statement
- Solution
 - Alert Generation - Early Fusion vs Late Fusion
 - Attack Technique Detection
 - Reinforcement Learning for Response
 - Action table
- Implementation
- Evaluation
- Schedule
- References

Motivation

- **Industrial Control System:**

- Critical infrastructure → High availability and safety requirement
- Integration between OT and IT → Larger attack surface

- **Challenge:**

- Cross level Attack Effecting Multiple Control Levels (Level 1 – Level 2)

Example: T0855 – Unauthorized Command Message

Compromise HMI (L2) → send malicious command to PLC (L1) → physical impact at L1 & Attacker Footprint at L2

- Detection Challenge: Single-level monitoring → miss cross-level attack propagation
- Response Challenge: Effective mitigation requires coordinated actions across multiple levels

- **AI Model Applied?**

- Stable normal data in OT → suitable for Unsupervised Learning (UL) anomaly detection
- Cross-level reasoning and context understanding → Large Language Model (LLM)
- Response to avoid overreaction – System Stability & Security Balance → Reinforcement Learning (RL)

Background: Industrial Control Systems (ICS)

- Industrial Control Systems (ICS)
 - Integration of hardware (OT) and software (IT)
 - Manages critical infrastructure processes
 - Architecture: follows Purdue Model (5 levels)
- Why OT vulnerable:
 - Rapid evolution, sophisticated attacks
 - Legacy systems
 - Performance-first priority, simple security protocols
 - Complexity, multi datasource

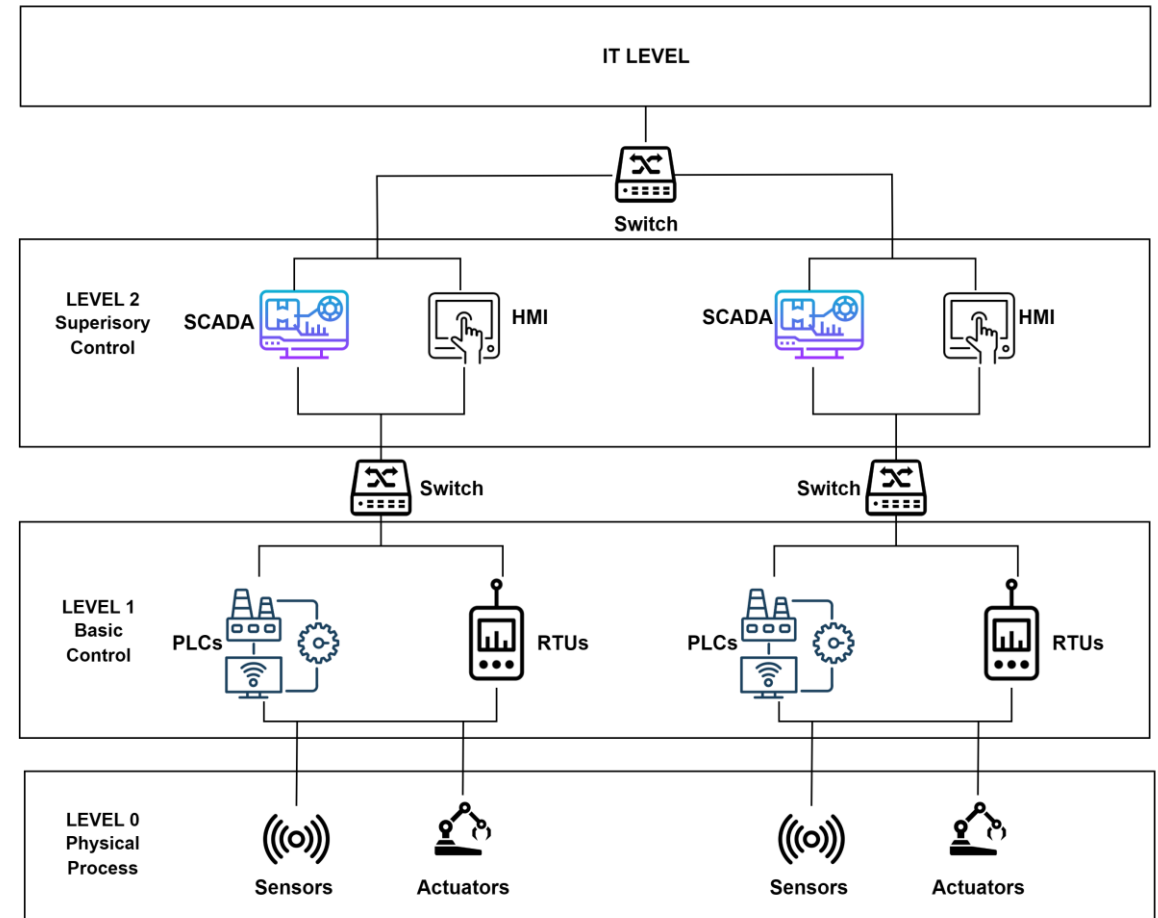


Figure 1. ICS Purdue Model

Background: MITRE ATT&CK Framework for ICS

- Knowledge base of adversary Tactics, Techniques, and Procedures (TTPs)
- Model attacker behavior, improving detection/response
- ICS Matrix
 - 12 Tactics, 83 Techniques
- Using detect and classify attacker activity for better response actions

Initial Access	Execution	Persistence	Privilege Escalation	Evasion	Discovery	Lateral Movement	Collection	Command and Control	Inhibit Response Function	Impair Process Control	Impact
12 techniques	10 techniques	6 techniques	2 techniques	7 techniques	5 techniques	7 techniques	11 techniques	3 techniques	14 techniques	5 techniques	12 techniques
Drive-by Compromise	Autotrun Image	Hardcoded Credentials	Exploitation for Privilege Escalation	Change Operating Mode	Network Connection Enumeration	Default Credentials	Adversary-in-the-Middle	Commonly Used Port	Activate Firmware Update Mode	Brute Force I/O	Damage to Property
Exploit Public-Facing Application	Change Operating Mode	Modify Program	Hooking	Exploitation for Evasion	Network Sniffing	Exploitation of Remote Services	Automated Collection	Connection Proxy	Alarm Suppression	Modify Parameter	Denial of Control
Exploitation of Remote Services	Command-Line Interface	Module Firmware		Indicator Removal on Host	Remote System Discovery	Hardcoded Credentials	Data from Information Repositories	Standard Application Layer Protocol	Block Command Message	Module Firmware	Denial of View
External Remote Services	Execution through API	Project File Infection		Masquerading	Remote System Information Discovery	Lateral Tool Transfer	Data from Local System		Block Reporting Message	Spoof Reporting Message	Loss of Availability
Internet Accessible Device	Graphical User Interface	System Firmware		Rootkit	Wireless Sniffing	Program Download	Detect Operating Mode		Block Serial COM	Unauthorized Command Message	Loss of Control
Remote Services	Hooking	Valid Accounts		Spoof Reporting Message		Remote Services	I/O Image		Change Credential		Loss of Productivity and Revenue
Replication Through Removable Media	Modify Controller Tasking			System Binary Proxy Execution		Valid Accounts	Monitor Process State		Data Destruction		Loss of Protection
Rogue Master	Native API						Point & Tag Identification		Denial of Service		Loss of Safety
Spearphishing Attachment	Scripting						Program Upload		Device Restart/Shutdown		Loss of View
Supply Chain Compromise	User Execution						Screen Capture		Manipulate I/O Image		Manipulation of Control
Transient Cyber Asset							Wireless Sniffing		Modify Alarm Settings		Manipulation of View
Wireless Compromise									Rootkit		Theft of Operational Information
									Service Stop		
									System Firmware		

Figure 3. MITRE ATT&CK for ICS

Background: Intrusion Detection Systems in OT

- Rule-based IDS: common tool IDS, NDR, firewall, ...
 - Analyze network packet.
 - Rely on pre-defined patterns, quick in known attack
 - Weak against zero-day attacks, misunderstand physical logic.
- Anomaly-based IDS
 - Machine learning classification.
 - Unsupervised Learning model prefer.
 - High false alert rate, hard to interpret/explain.

Background: AI Agent

- **Agent:**
 - Autonomous entity that observes, reasons and acts
 - Properties: adaptive, autonomous, independent decision-making
- Fits ICS security:
 - Autonomous detection-response
 - Operates indepently
 - Learning system-specific behavior

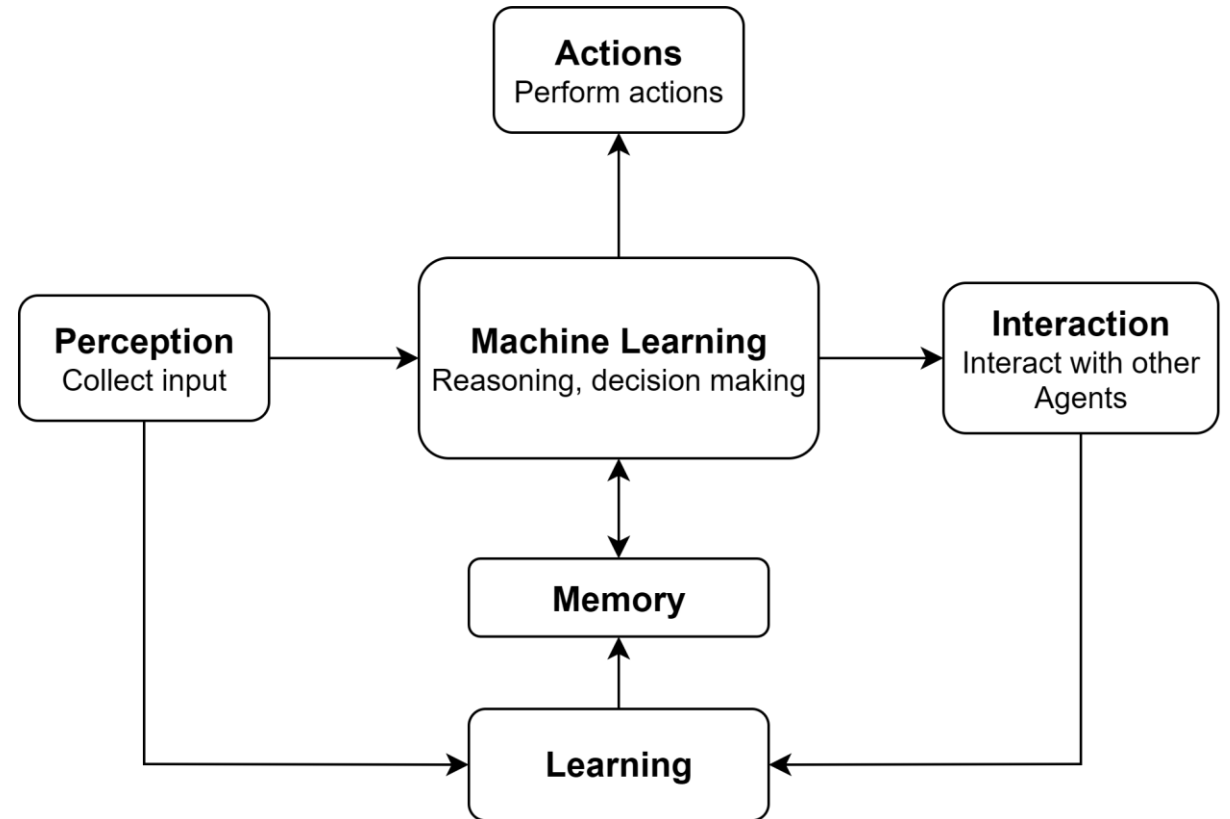


Figure 4. AI Agent Architecture

Background: Reinforcement Learning

- Reinforcement Learning (RL)
 - State (S_t): Current state of environment
 - Action (A_t): Decision made by the agent
 - Reward (R_t): Feedback based on effectiveness of the action
- Objective: Maximize cumulative rewards over time

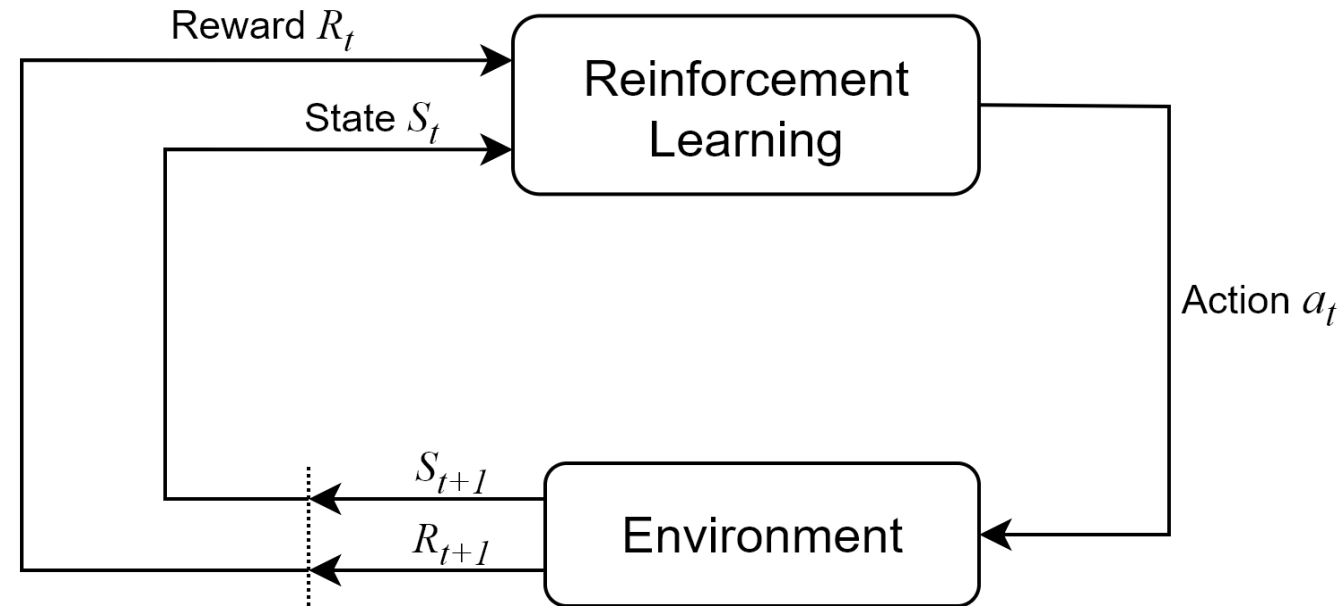


Figure 6. Agent-Environment interaction in RL loop

Related Work on AI-based Detection and Response in ICS

Target	Paper	OT Level Coverage		AI Model	Detection capability		Response capability		Cross – Level Reasoning	Testbed environment
		L1	L2		Normal/ Abnormal	Attack Classification	Security	Stability		
Detection	[1] 2024	O	O	SL	O	-	-	-	O	SWaT
	[2] 2025	O	-	UL	O	-	-	-	-	Ladder Logic Bombs for PLC logic simulation
	[11] 2025	-	O	Semi-supervised Deep Learning	O	O	-	-	-	Public ICS dataset
Response	[3] 2022	-	O	RL	-	-	O	O	-	ICS network simulation
	[4] 2024	O	O	RL	-	-	O	-	O	Power grid voltage control CPS
	[12] 2023	-	O	Deep RL	-	-	O	O	-	GNS3 Network Emulation
Detection & Response	[5] 2024	O	-	Rule Based + RL	O	-	O	O	-	Power grid / electric CPS
	[6] 2025	O	-	Supervised CNN + RL	-	O	O	O	-	Industrial Internet of Things (IIoT)
	[7] 2025	O	-	RL	O	-	O	O	-	IoT – Smart factory Testbed
	[10] 2025	-	O	CNN + LSTM + Deep RL	O	O	O	O	-	Public IoT dataset
	Ours	O	O	UL + Transformer + RL	O	O	O	O	O	ICS Simulation – Water Treatment,

Table 2. Related work on Detection and Response in ICS

Issues

- Cross-level vs Single-level Detection and Response
 - How does a cross-level agent that jointly reasons over multiple OT layers compare to single-level agents in terms of attack detection quality and response to block the attack?
- Attack technique Detection
 - How effectively can the Transformer-based module could classify the technique compared to other models?
 - How accurate is the unsupervised anomaly detection (UL) model on ICS/OT data?
- RL Agent State Space Comparison: Big vs Small
 - How does Physical System State effect RL Action to avoid overreaction

Notations (1/2)

Category	Symbol	Term
Data source	$\mathcal{D} = \{D_w^l w \in \{1, 2, 3\}, l \in \{1, 2\}\}$	Data source from multi source OT-level
	w	Source Type index: $w = 1$ – sensor, $w = 2$ – traffic, $w = 3$ – logs
	l	OT Level index: $l=1$ – Level 1, $l = 2$ – Level 2
	$d_{w,t}^l$	Raw data sample from source w and level l in step t
	$f_{w,t}^l$	State feature extracted from raw data $d_{w,i}^l$ by Perception module
Alert event	$e_{w,t}^l$	Anomaly alert generated by the unsupervised learning model from source w and level l at step t
	e_t^{corr}	Correlated alert compiled from multiple raw alerts $e_{w,i}^l$ across different data sources and levels at step t
Classification Module	$N_{\mathcal{T}}$	Number of predefined techniques
	$\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_{N_{\mathcal{T}}}\}$	Set of predefined attack techniques
	$\hat{\tau}_t \in \mathcal{T}$	Mapped attack technique for alert e_t^{corr}

Notations (2/2)

Category	Symbol	Term
Response Module	$N_{\mathcal{A}}$	Number of predefined response actions
	$\mathcal{A} = \{a_1, a_2, \dots, a_{N_{\mathcal{A}}}\}$	Set of predefined response actions
	$x_{m,t}^l, m \in \{1, 2, 3, 4, 5\}, l \in \{1, 2\}$	OT environment state m at level l and step t $m=1$ – device state, 2 – packet rate, 3 – tank overflow flag, 4 – unreachable device flag, 5 – isolated device flag;
	m	OT environment state index: $m=1$ – device state, 2 – packet rate, 3 – tank overflow flag, 4 – unreachable device flag, 5 – isolated device flag
	$s_t = (\hat{t}_t, \{e_{\{w,t\}}^l\}_{\{w,l\}}, \{x_{\{m,t\}}^l\}_{\{l,m\}})$	cross-level RL state at step t , combining detected technique $\hat{t}_t$, multi-source UL alerts $e_{w,t}^l$, OT environment state $x_{m,t}^l$
	$a_t \in \mathcal{A}$	Response action selected by RL agent for state s_t
	r_t	OT-environment reward for action a_t
	$\pi(a_t s_t)$	Policy function mapping system state to an action.
	$Q_{\theta}(s_t, a_t)$	Tabular Q-value for state-action pair
	$done_t$	Episode Terminal Flag

Problem Overview

- ICS Kill Chain follows multiple steps and stages, effecting multiple components & control level.
- How AI Agents can identify attack techniques (detect) and perform actions (response) to stop the Kill Chain progression?

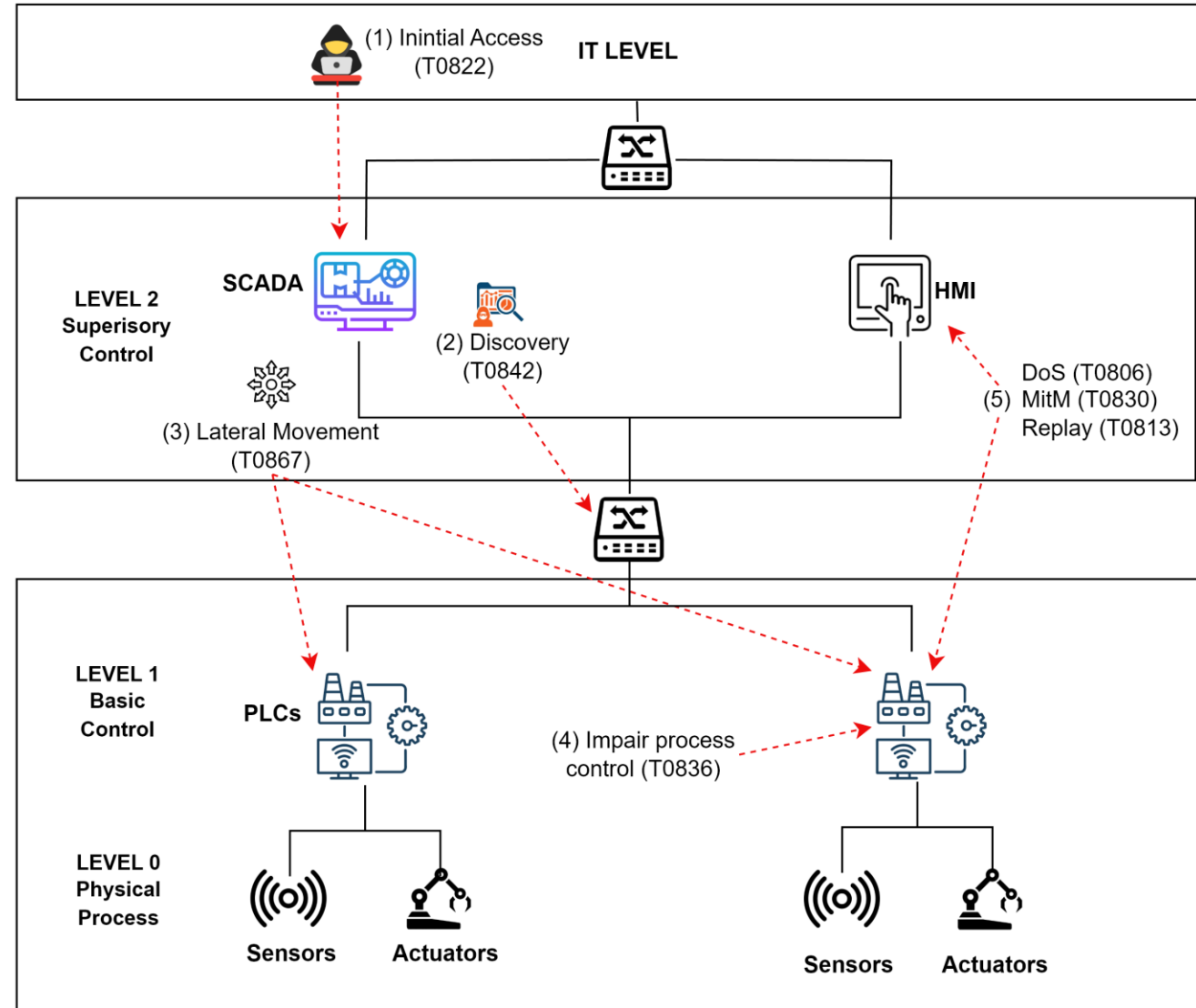
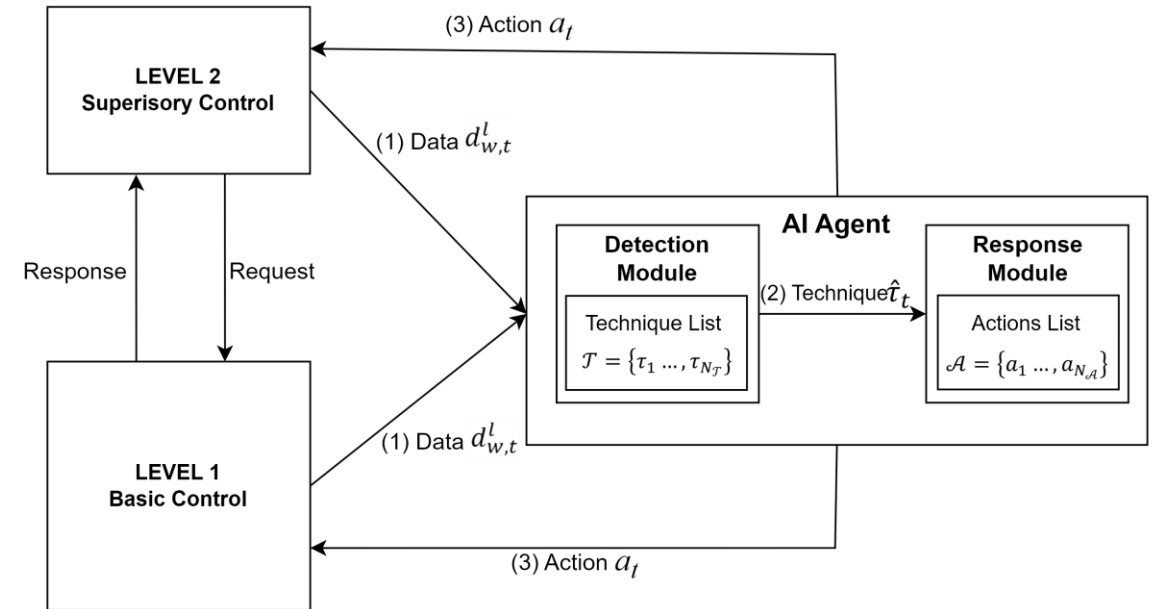


Figure 6. Example Kill Chain in ICS-OT

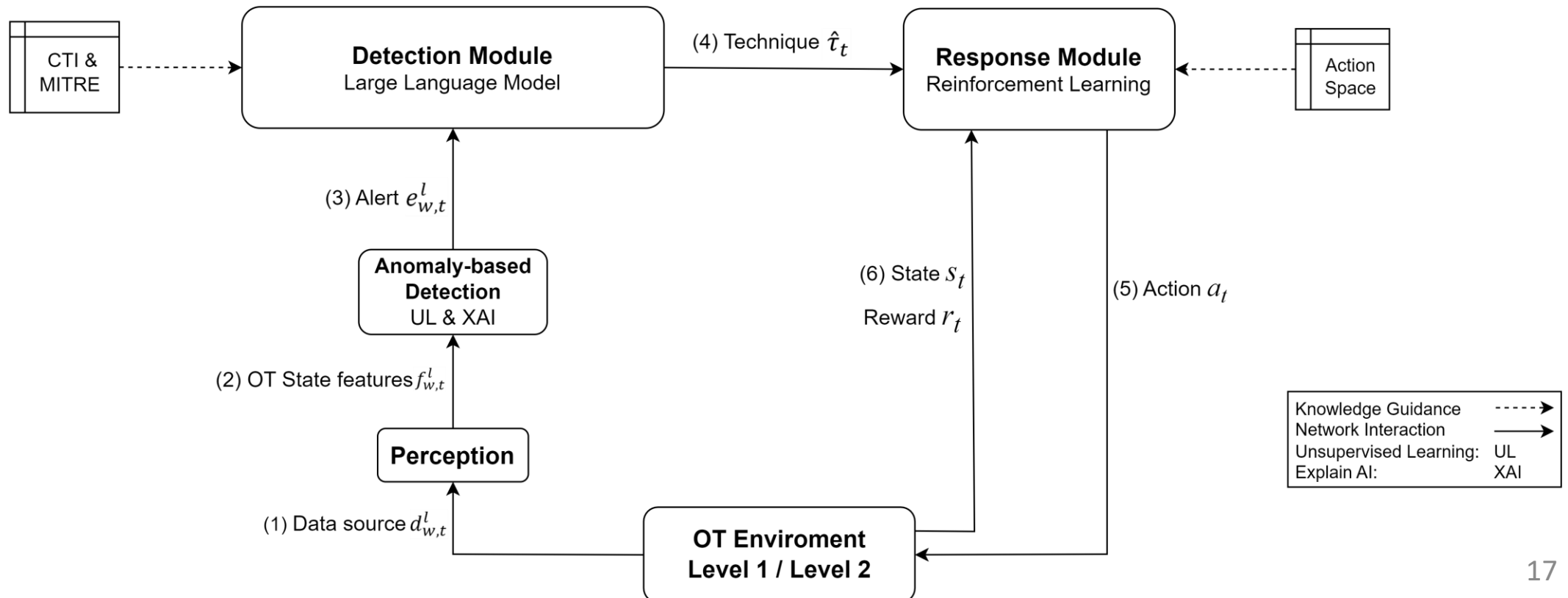
Problem Statement

- Input:
 - OT-level data $\mathcal{D} = \{D_w^l | w \in \{1, 2, 3\}, l \in \{1, 2\}\}$
 - Level 1: sensor, traffic, PLC logs
 - Level 2: traffic, HMI logs
 - Predefined Attack technique $\mathcal{T} = \{\tau_1 \dots, \tau_{N_T}\}$
 - Predefined actions: $\mathcal{A} = \{a_1 \dots, a_{N_A}\}$
- Output:
 - Attack technique $\hat{\tau}_t$
 - Response action a_t
- Objective:
 - Maximize accuracy of detected Technique $\hat{\tau}_t$
 - Minimize the appearance possibility of the next Technique $\hat{\tau}_{t+1}$ and System Disruption
- Constraint:
 - Predefined Attack technique $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_{N_T}\}$
 - Predefined actions $\mathcal{A} = \{a_1, a_2, \dots, a_{N_A}\}$



Solution: AI Architecture

- Perception module: Preprocess Multiple-Level raw OT data sources into meaningful features
- Anomaly-based detection: raise anomaly alert to Detection Agent
- Detection Module: Aggregate Multi-level Alerts into a Cross-level Alert, classify attack technique
- Response Module: Action decision Across Both Level



Solution: Attack Technique Detection

- Correlation Module: Aggregate multi-source, multi-level Anomaly Alerts into a Cross-level Correlated Alert.
- Retriever: Select Candidate Techniques $\mathcal{T}_t^{(r)} = \{\tau_{t,1}^{(r)}, \tau_{t,2}^{(r)}, \dots, \tau_{t,N_r}^{(r)}\}$ from Predefined Attack technique $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_{N_T}\}$
- Reasoning Module: Reasoning and select the most likely Technique $\hat{\tau}_t$

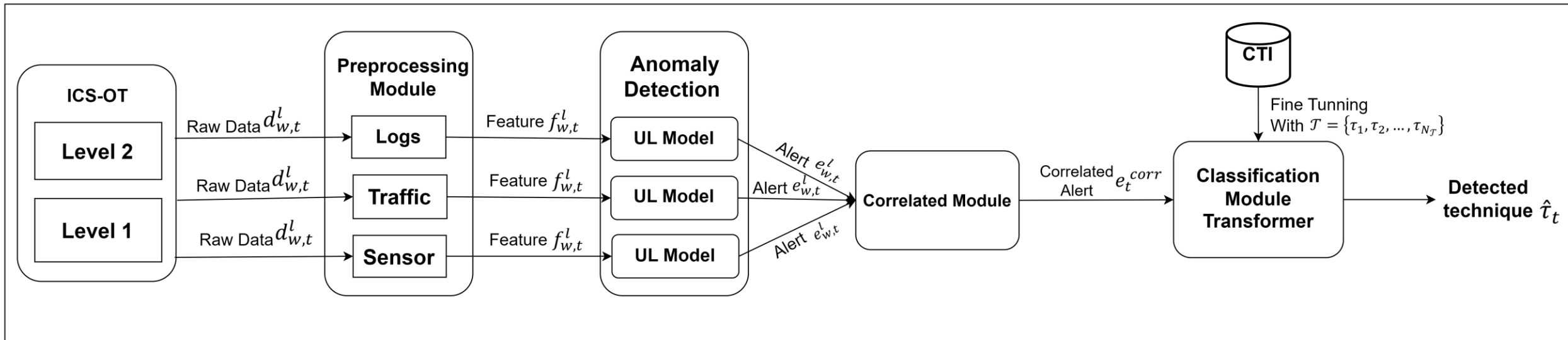


Figure 9. Attack Technique Classification Architecture

Solution: Response Action Selection

- State Definition $s_t = (\hat{\tau}_t, e_{w,t}^l, x_{m,t}^l)$ (2 sources):
 - Detection module: Detected Attack Technique $\hat{\tau}_t$ & UL anomaly signals $e_{w,t}^l$
 - Current physical system state $x_{m,t}^l$ (HMI/PLC Status, tank overflow flag,...)
- Select action a_t at Both Level based on predefined action space
- Reward factors r_t :
 - Security: whether Attack $\hat{\tau}_t$ is successfully stopped
 - Stability: presence/absence of process disruption

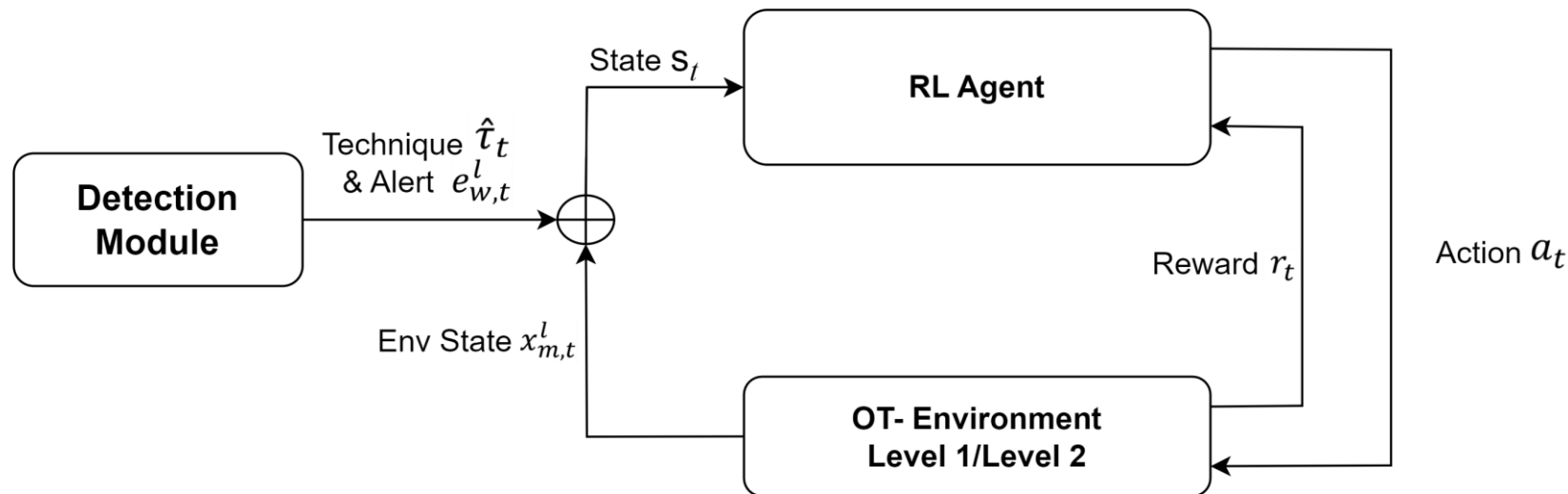


Figure 10. RL-based Incident Response Architecture

Implementation: Overview

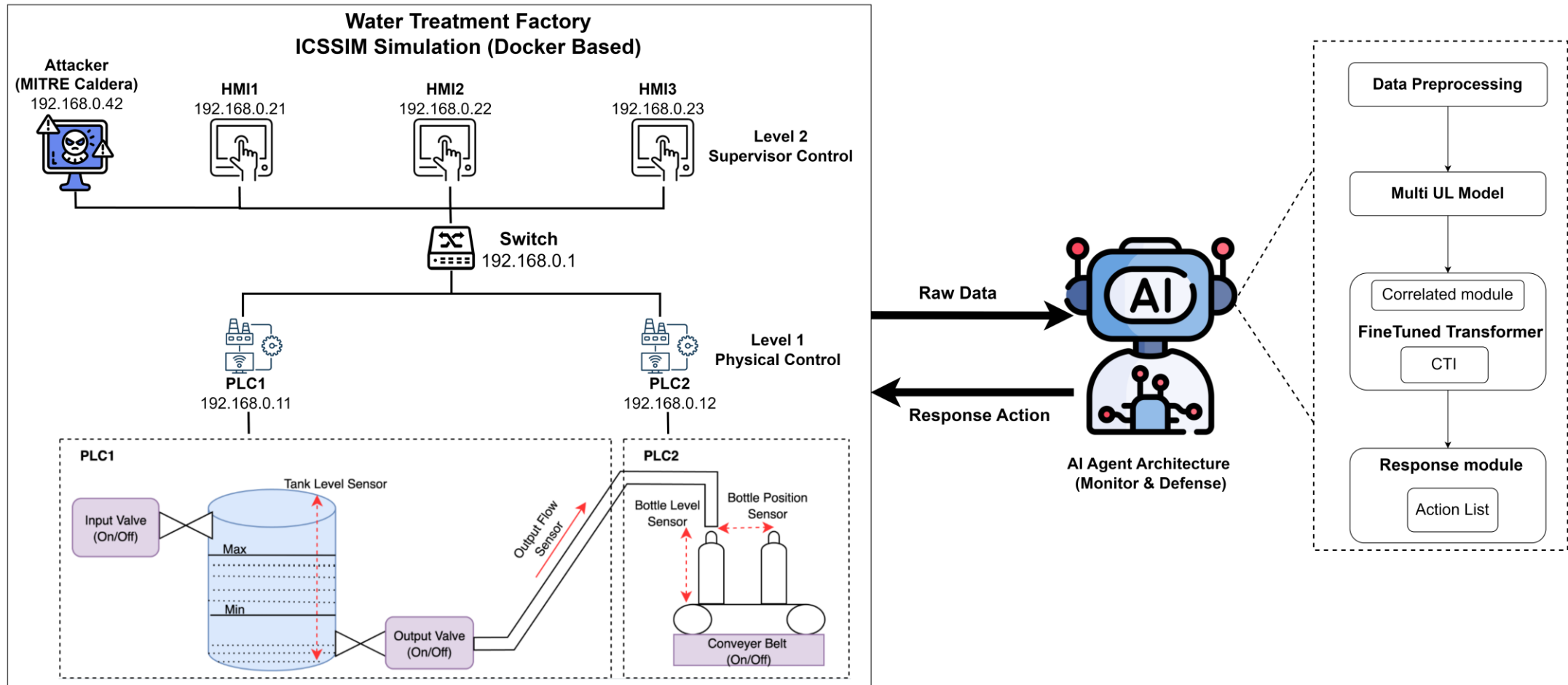


Figure. Implementation Overview

Implementation: Open-source Tools & Libraries

Category	Name	Functionality
Tools	MITRE Caldera for OT	Attack generation
	ICSSIM	ICS environment deployment
	Docker	Containerize testbed
	ICSFlow	Network Traffic Flow Extraction
	Wireshark	Collect & Analyze Network Data
	Rsyslog, Tcpdump	Syslog & Network Traffic Recorder
	Suricata	IDS tool – signature-based detection
Python Libraries	Scapy	ICS protocol modification
	Scikit-learn	Machine learning
	PyTorch/RLlib	Implementing of RL algorithm
	OpenAI API	Alert-correlation & Technique mapping
	Matplotlib, Pandas, Seaborn	Visualisation & Data Analysis

Table 4. Open-source Tools & Libraries

Implementation: Attack List

Stage	ID	Name	Description
1	T0840	Network Connection Enumeration	Capture network traffic to observe communications and extract Ip address, hostname, devices properties.
2	T1110	Brute-force SSH	Repeatedly attempt passwords to gain unauthorized SSH access.
3	T0864	Transient cyber access	Login the system with unauthorized session
4	T0892	Change Credential	Modify authentication data (password)
5	T0886	Exploit SSH Remote Service Lateral Movement	Exploiting SSH services to deploy attacker agent on multiple devices
6	T0830	Man in the Middle	Intercept and alter communications between HMI and PLC
7	T0861	Point and Tag Identification	Identify control tags used in sensor/actuator
8	T0836	Modify Parameter	Change control parameters of PLC: max/min tank value
9	T0855	Unauthorized Command Message	From HMI send unauthorized control commands to PLC
10	T0803	Block Command Message	From PLC, block legitimate control commands of HMI
11	T0814	Denial of Service	Flooding traffic into PLC devices with hundred read command per second

Table 5. Attack List

Implementation: Action Table

Action	Description	Risk	Level
Alert	Send Alert to SOC	Low	L2/L1
Terminate SSH Session	Forcefully terminate active SSH sessions suspected to be used by attackers	Low	L2/L1
Remove Attacker File	Remove malicious files that attacker agents deployed during lateral movement stage	Low	L2
Rate Limit	Limit traffic in/out devices to reduce excessive requests or flooding	Medium	L2/L1
ARP Table block	Preventing Device to update a new ARP Entry	Medium	L2/L1
Disable Write Request	Disable write operations to prevent attackers from modifying PLC registers or control parameters.	Medium	L1
Disable User Account	Temporarily disable compromised or suspicious user accounts.	Medium	L2
Change Credential	Reset password	Medium	L2/L1
Block Read request	Block unauthorized or suspicious read requests to PLC registers	High	L1
Modify Physical Parameter	Restore/Adjust manipulated physical tank/bottle level values to normal values	High	L1
Terminate HMI Session	Terminate suspicious or unauthorized HMI user sessions	High	L2
Restore System State	Restart devices	Critical	L2
Isolate device	Disconnect a compromised device from the network	Critical	L2/L1

Evaluation

1. Cross Level vs Single Level Comparison

- Detection rate (TPR): % of attacks correctly detected
- Response success rate: % of attacks successfully prevented

2. Detection Module Performance

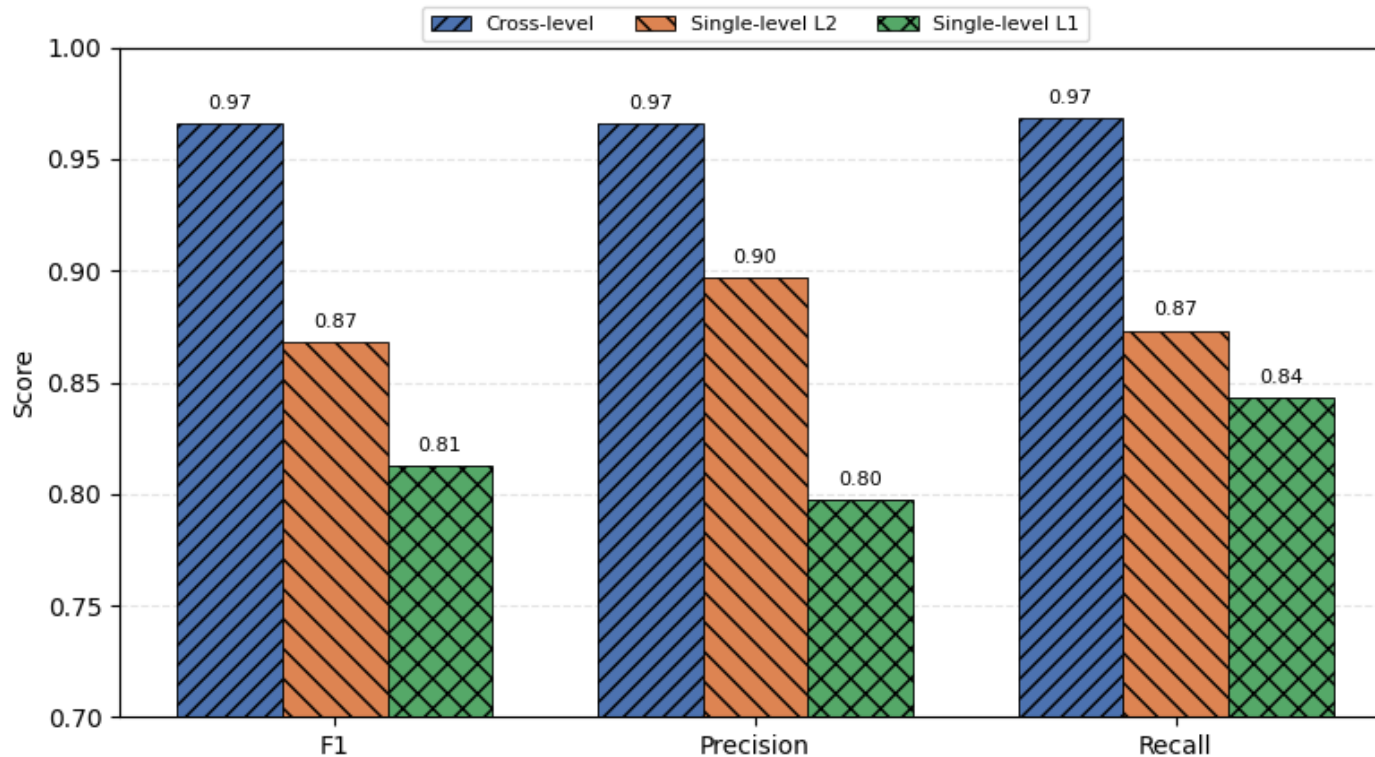
- Classification Module
 - LLM vs Transformer: compare F1/Precision/Recall and Inference time
- Anomaly Detection
 - Multidata source vs Single Source

3. Response Module Performance

- State space - Big vs Small:
 - Reward curve to show learning trend
 - Selected action to show how physical system state effect Agent decision

Evaluation 1: Cross-Level vs Single-Level Agent Detection

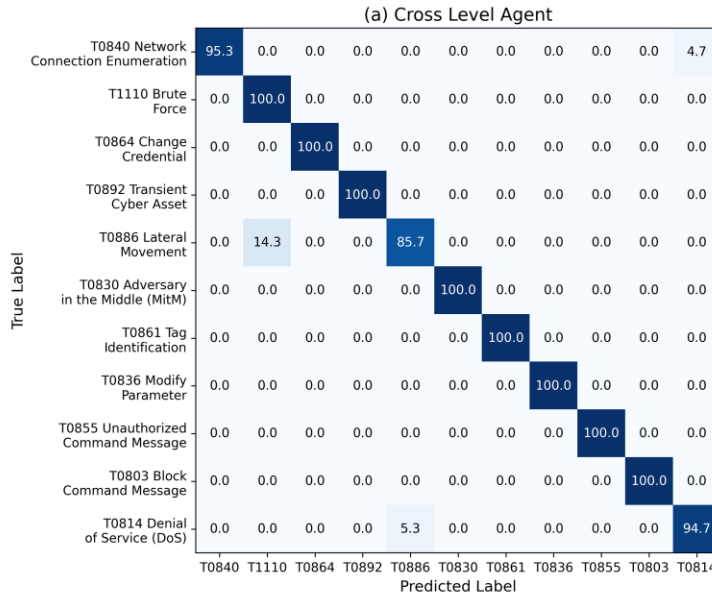
Higher Overall F1 Score, Precision, Recall of Cross-Level



- **Best overall:** Cross Level Agent - F1 / Precision / Recall ~ 0.97
 - Highest performance across all metrics
 - Cross Level Agent achieve Full attack visibility
- **Single-level gap:** $L2 > L1$ on all metrics
 - Level 2 Logs contain important attack traces

Evaluation 1: Cross-Level vs Single Level Agent Detection

Cross Level Agent is robust cross all technique

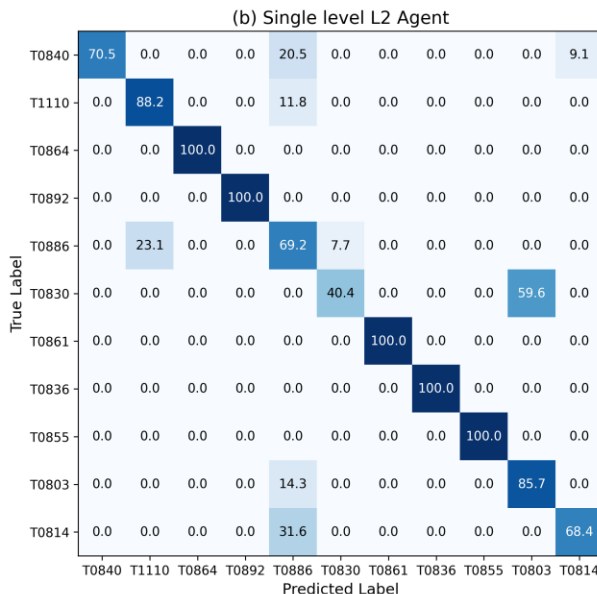


Cross-level agent

- Robust across almost all techniques
- Most detection rates stay above 90%

Single-level agents: Struggle on specify techniques

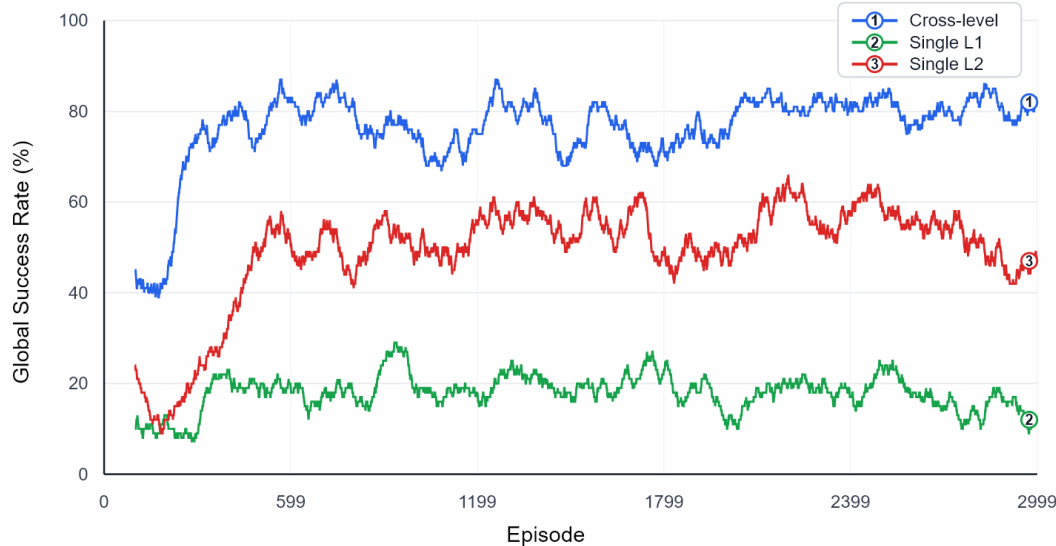
- L2 confuses T0830 vs T0803 (59.6%)
→ Both make the HMI unable to read PLC data, require PLC-side signals
- L1 confuses T0836 vs T0855 (88.1%)
→ Both cause similar physical effects at L1, require HMI pattern to classify



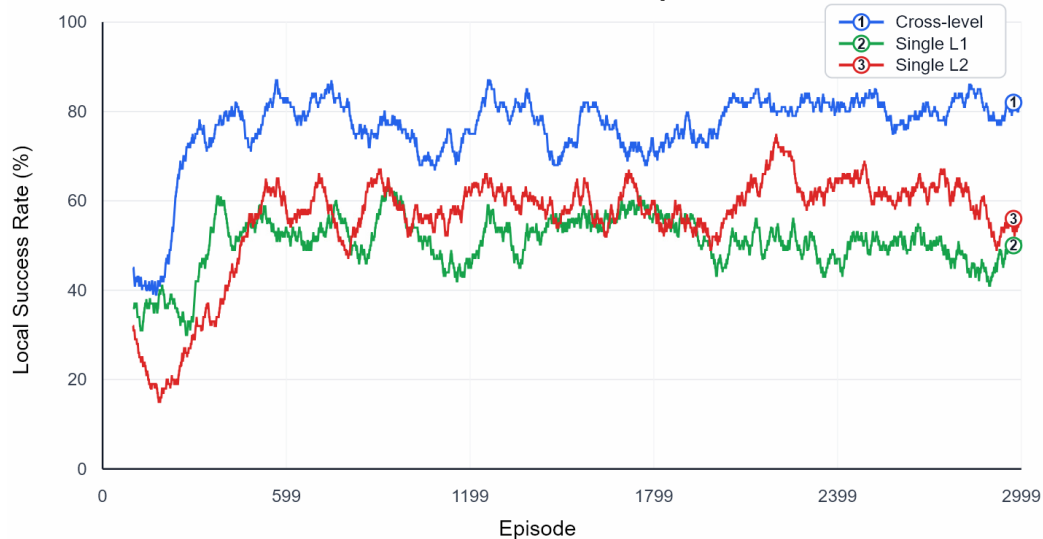
Evaluation 1: Cross-Level vs Single Level Agent Response

Higher Attack Mitigation Success Rate Of Cross-Level Agent

Global Success Rate Comparison



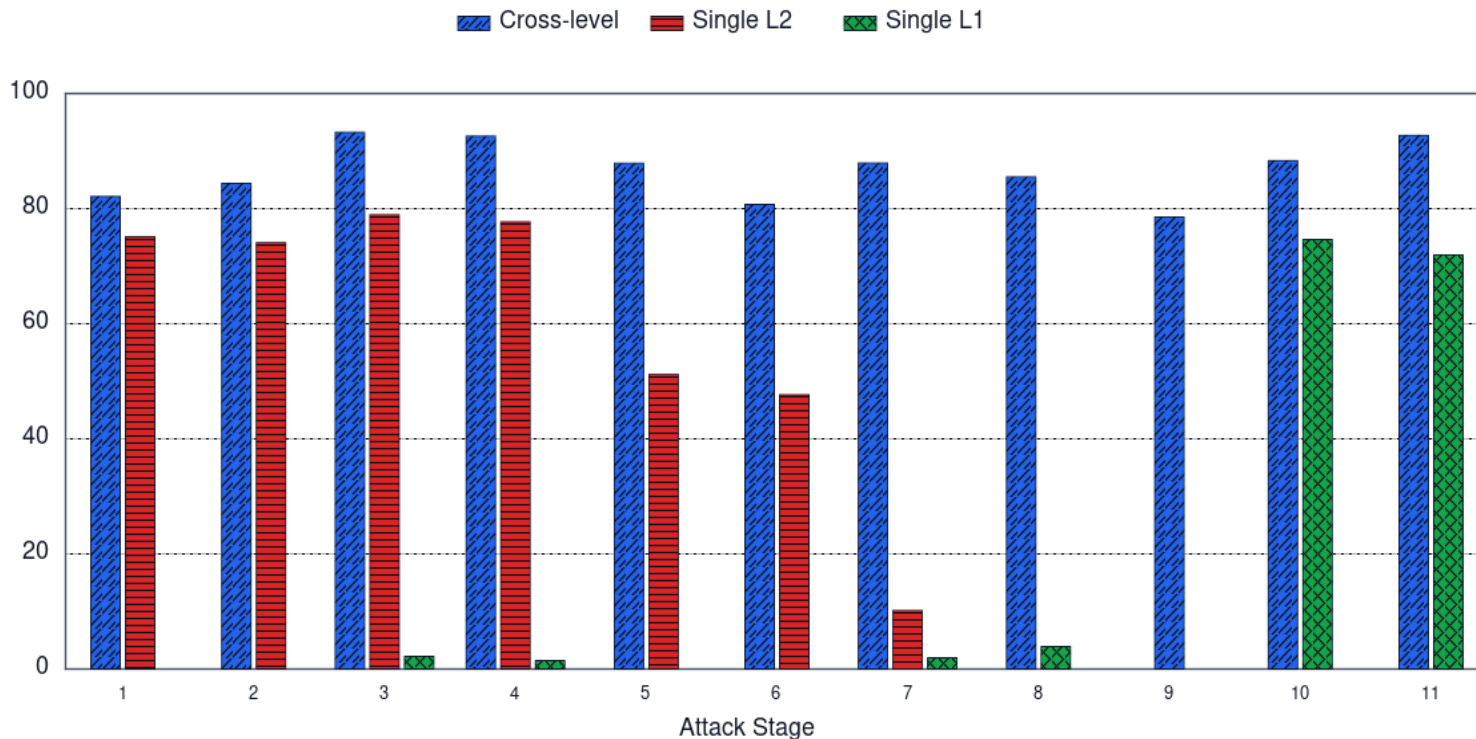
Local Success Rate Comparison



- **Global success:** attack totally stopped across the whole system
 - Cross-level – Best ~80%, 20% higher L2, 60% higher L1
 - Coordinated response across levels improves system-wide mitigation
- **Local success:** Agent could recover abnormal signal from its own level
 - Both single-level agents improve locally; L2 ~60%, L1 ~50%, but lower than Cross-Level Agent
 - Per-level protection, but not full-system recovery

Evaluation 1: Cross-Level vs Single Level Agent Response

Cross-level is robust; single-level is stage-dependent.



- **Cross-level** ~80–90% across all stages
→ Stable response throughout the full attack chain
- **Single-L2**: Strong at early--middle stages, Drops after Stage 5-6
Why: Early--middle stages originate at L2; by Stage 6 (lateral movement), the attacker has C2 across system
- **Single-L1**: rises at late impact stages
Why: Stage 10 – 11: Impact Stage, target on L1

Evaluation 1: Action Selected by RL Module

Adaptive Action Selected by Cross-level Agent

ID	Attack	Alert	Terminate SSH session	Remove Attacker file	Rate limit	Arp block	Disable write request	Disable user account	Change credential	Block read request	Terminate HMI session	Modify physical parameter	Isolate device	Restore system state
T0840	Network Connection Enumeration	3.03	2.57	2.48	2.83	2.51	2.62	2.59	2.59	-2.38	2.1	2.2	3.83	-0.54
T1110	Brute-force SSH	8.68	9.65	8.16	8.69	8.04	8.32	8.09	8.39	2.57	5.69	6.66	5.43	4.35
T0864	Transient cyber access (Login)	6	5.46	5.46	5.87	5.29	5.77	6.2	6.08	3.14	5.17	4.67	2.41	3.33
T0892	Change Credential	5.82	5.22	5.35	5.61	5.12	5.53	5.96	5.76	2.77	5.43	4.67	1.63	2.97
T0886	Exploit SSH Remote Service Lateral Movement	9.24	10.27	8.91	9.02	8.51	8.71	8.32	8.25	3.38	6.87	6.05	5.89	6.86
T0830	Man in the Middle	3.99	3.57	3.34	3.68	3.59	3.53	3.51	3.54	-0.55	3.12	2.69	4.79	1.45
T0861	Point and Tag Identification	2.97	2.2	2.37	2.61	2.28	2.46	2.16	2.37	0.71	2.15	1.42	-0.73	0.57
T0836	Modify Parameter	3.92	3.46	3.28	3.73	3.41	3.97	3.37	3.63	2.89	3.33	6.13	-0.37	1.57
T0855	Unauthorized Command Message	4.04	3.57	3.48	3.87	3.69	3.91	3.53	3.63	2.85	3.79	6.27	-0.07	2.18
T0803	Block Command Message	4.65	4.34	4.55	5.05	5.02	4.78	4.78	4.9	4.68	4.6	4.55	4	5.3
T0814	Denial of Service	8.09	7.55	8.17	7.91	7.56	7.6	7.78	7.73	6.4	7.11	1.29	5.2	6.68
Avg Best Q-Value		5.58	5.26	5.05	5.35	5	5.2	5.12	5.17	2.41	4.49	4.24	2.91	3.16

Table 6. Q-Table Technique vs Actions

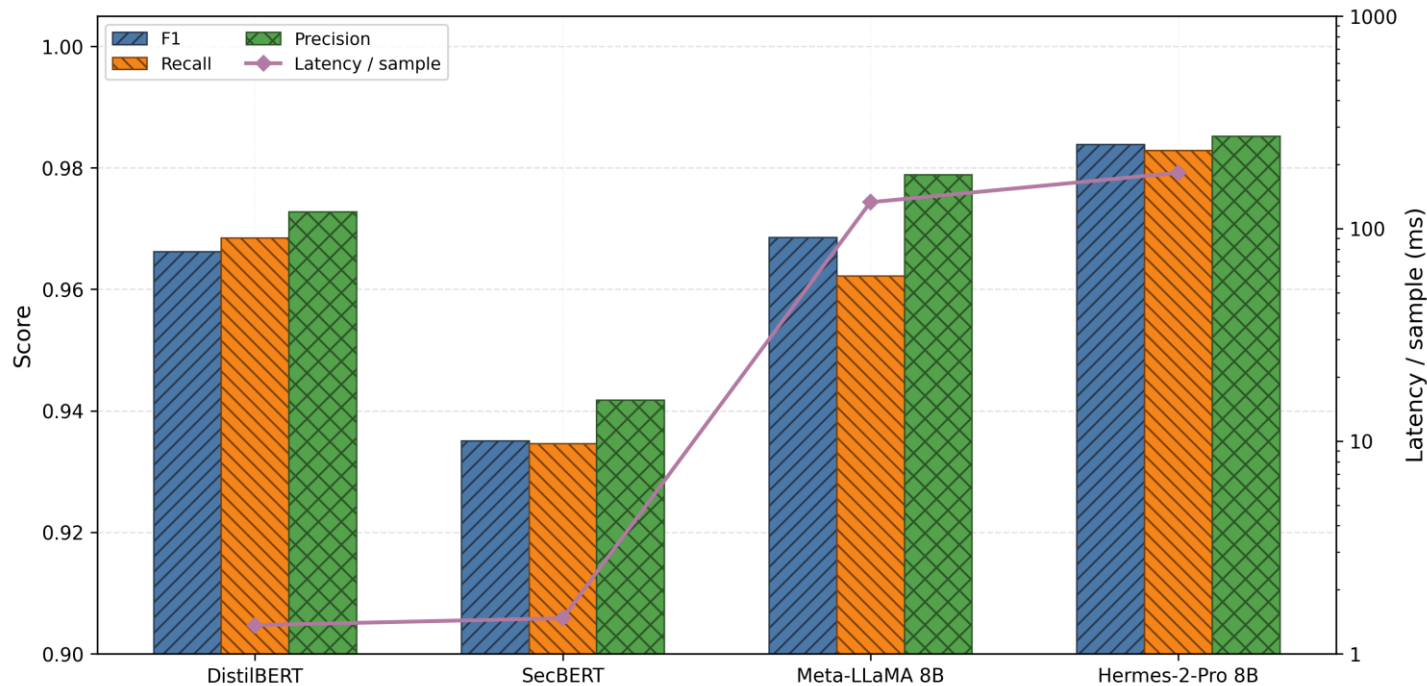
- **Best-Q spread:** distributed best action across multiple action: alert, SSH termination, credential change, parameter modification,...
- **Isolate device**
Best-Q for 3 techniques, but average Q only ~2.91
→ High impact, high penalty
- **Why:** Reward logic → effective actions are rewarded, but also penalizing unnecessary critical response
→ Adaptive and reasonable learned policy, selected actions match attack context

Evaluation 2: Transformer vs LLM in Classification

Slightly higher performance but much slower LLM

Table: Performance & Latency

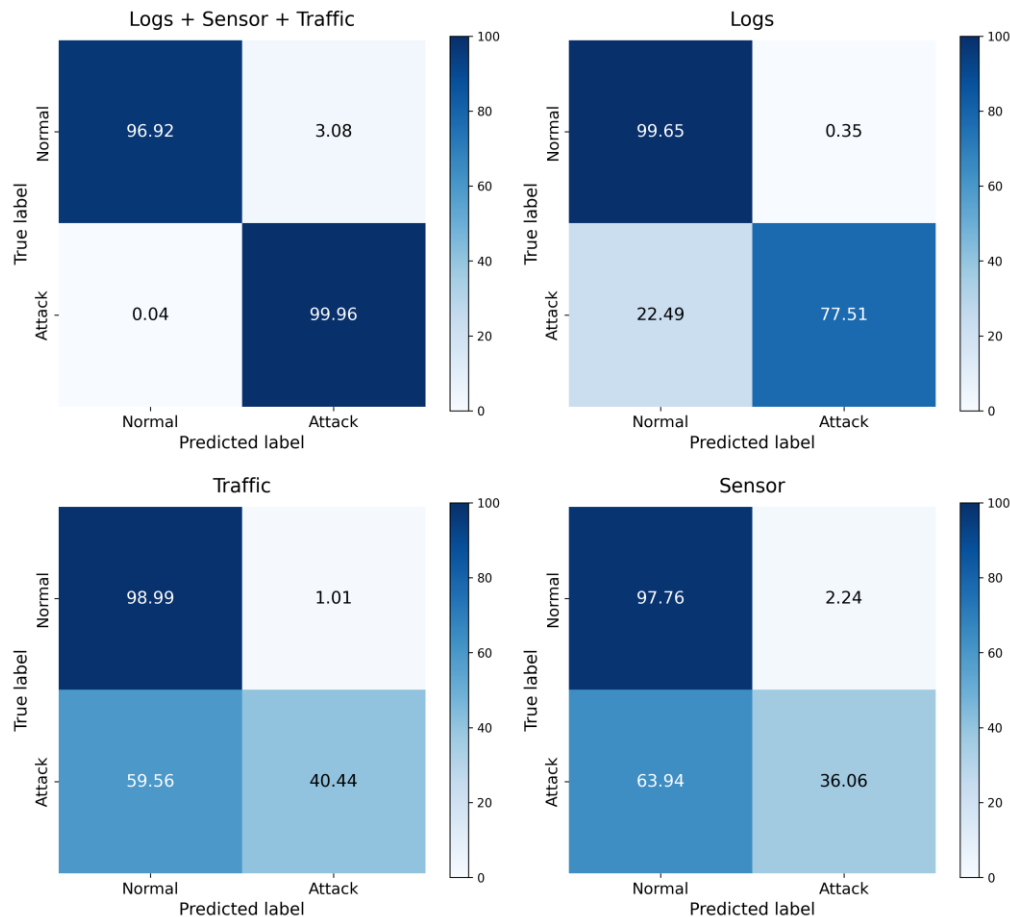
Model		F1	Recall	Precision	Latency (ms)
Transformer	DistilBERT	0.9662	0.9685	0.9728	1.36
	SecBERT	0.9351	0.9346	0.9418	1.47
LLM	Meta-LLaMA 8B	0.9686	0.9623	0.9789	133
	Hermes	0.9839	0.9829	0.9852	183



- **Accuracy:** Hermes (LLM) ≈ 0.984 F1 > DistilBERT (Transformer) ≈ 0.967 (small gain).
- **Latency:** Transformers $\sim 100\times$ faster ($\approx 1.3\text{--}1.5$ ms/alert) than LLMs ($\approx 133\text{--}183$ ms/alert)
- Transformers offers a better accuracy–latency trade-off
- LLM are more suitable for post-hoc analysis

Evaluation 2: Anomaly Detection Performance Across Multi Data source

Multi-source detects best, logs are the best single source

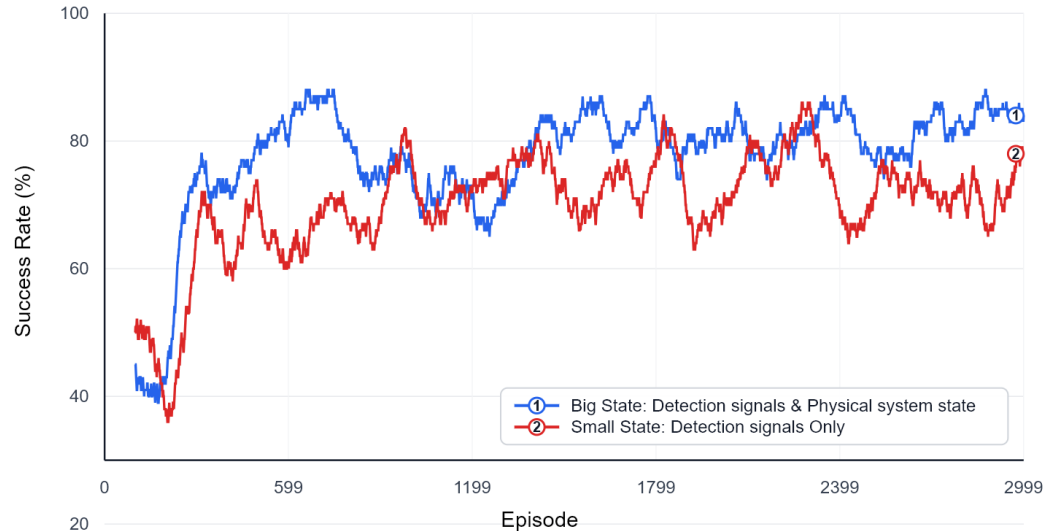


- **Best detection:** Multi source Fusion ($\sim 99.96\%$ attack recall)
- **Why:** Any source fires $\rightarrow$ flagging whole window $\rightarrow$ Reduce false negatives
- **Single-source gap:** Logs $\sim 77.51\%$; Traffic $\sim 40.44\%$; Sensor $\sim 36.06\%$
- **Why:** Logs contain richer attack pattern than the others
- **Normal behavior:** All sources keep high normal recognition ($\sim 98 - 99\%$) $\rightarrow$ UL learns the normal baseline effectively

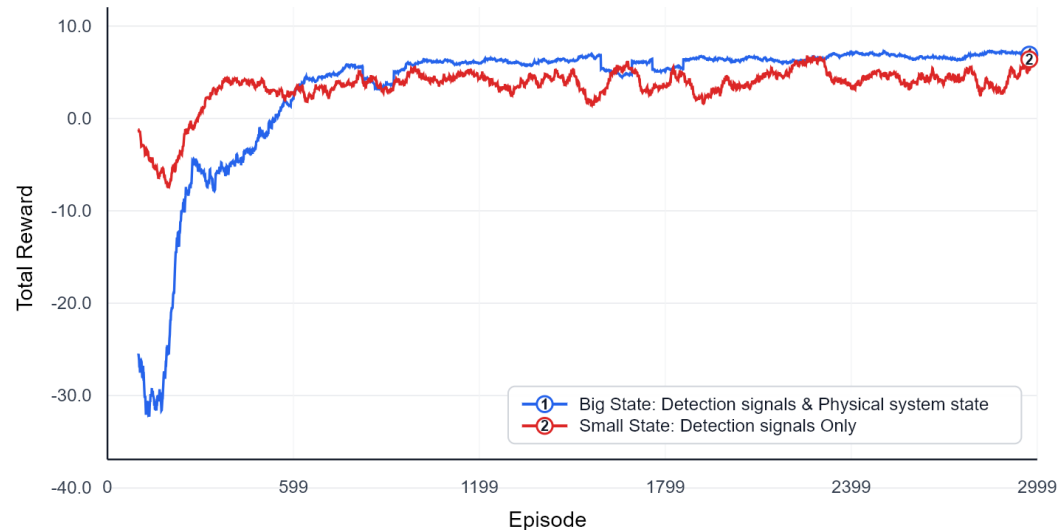
Evaluation 3: State Space - Big vs Small (1/2)

Bigger State-Space learns slower, but finishes stronger and more stably

Success Rate Comparison



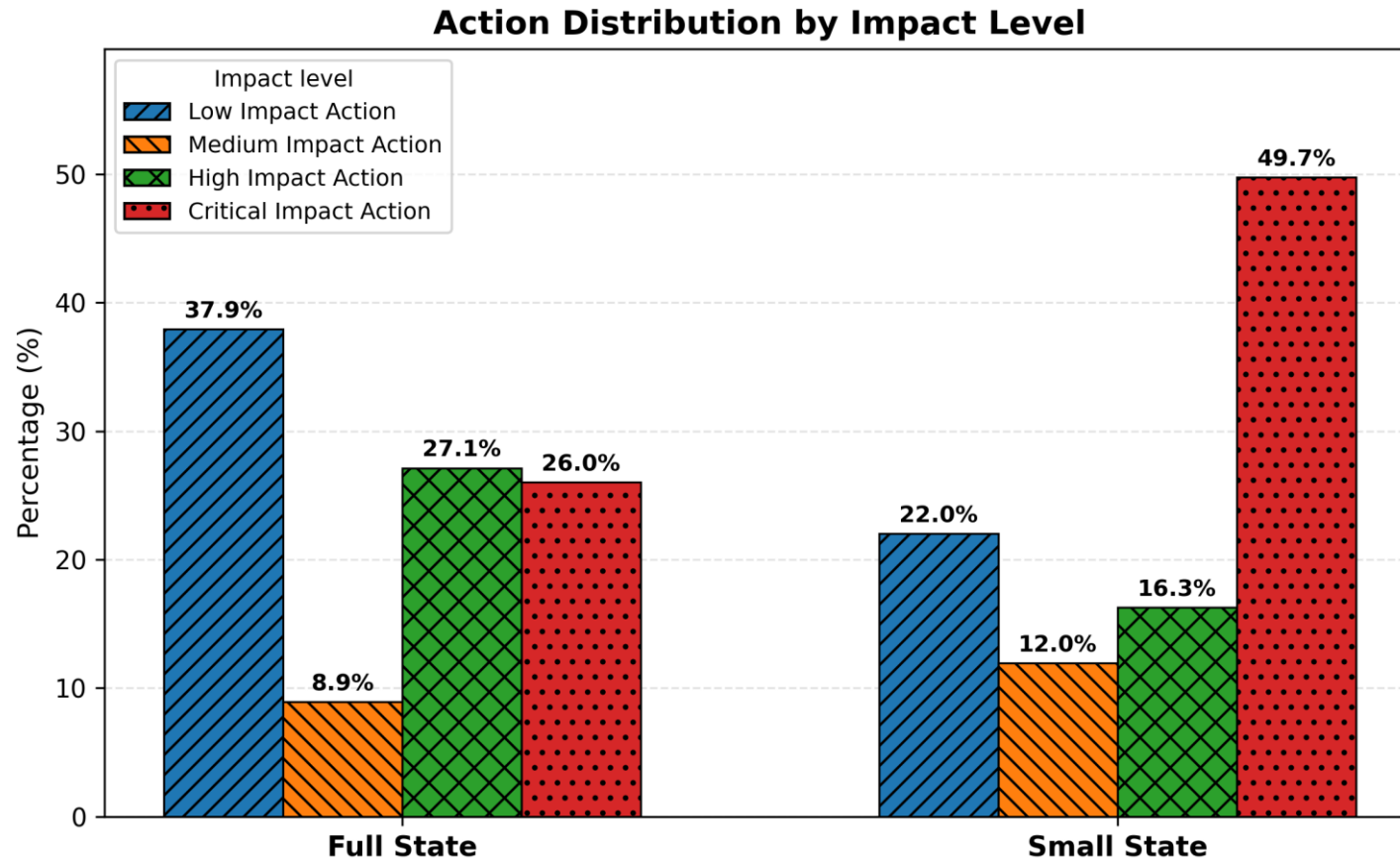
Reward Curve Comparison



- **Success Rate:**
 - Big state achieve 5% higher → better action quality
 - **Why:** Adds physical-system context → More informed response decisions
- **Reward:**
 - Full state converges later (~600 vs 300 episodes) but shows more stable learning trend
 - **Why:** Big state includes more state → increases dimensionality

Evaluation 3: State Space - Big vs Small (2/2)

Full State helps Agent learn to avoid over-reaction



- **Full State:**
 - More low-impact actions (~37.9%)
 - Action choices are more evenly balanced (38%, 27%, 26%)
- **Small State:**
 - Critical actions dominate (~49.7%)
- **Why:** in small state, agent is free to choose high/critical impact actions that have a higher probability of reducing anomalies.

References

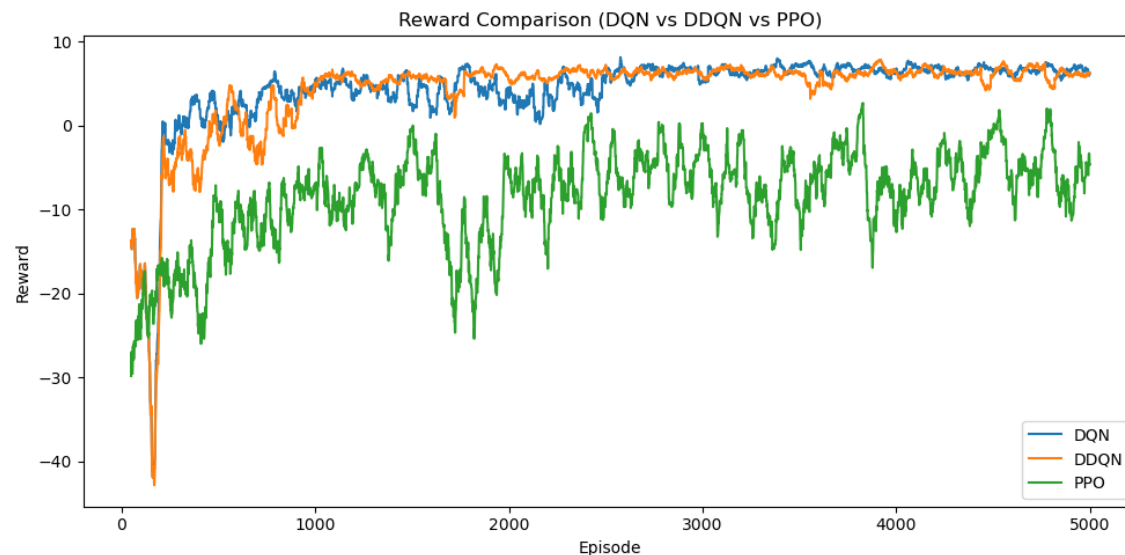
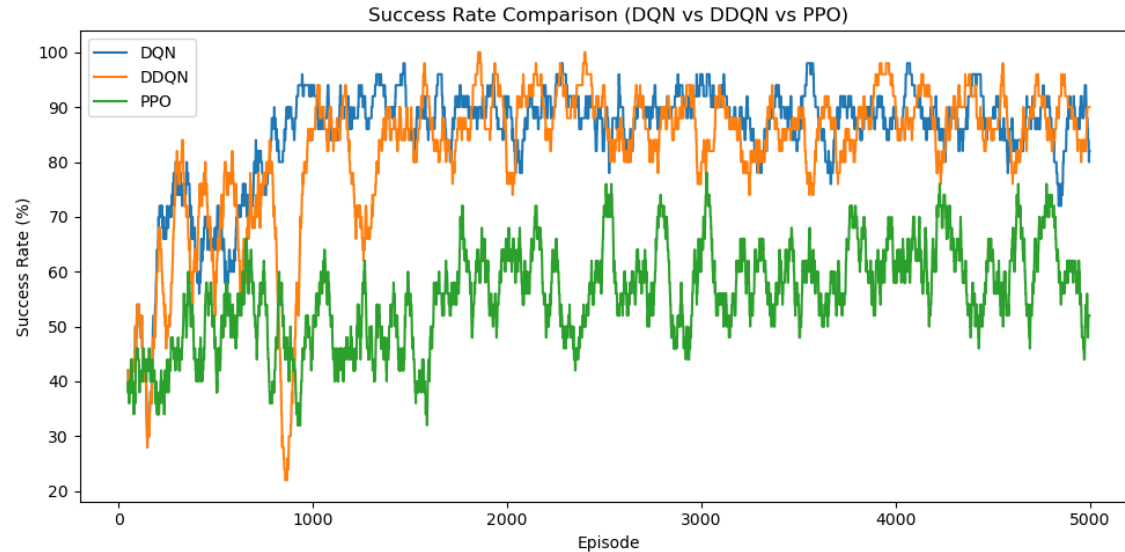
- [1] V. Berge and C. Li, “Enhanced Anomaly Detection in Industrial Control Systems aided by Machine Learning,” arXiv preprint arXiv:2410.19717, Oct. 2024. [Online]. Available: <https://arxiv.org/abs/2410.19717>
- [2] J. H. Lee, I. H. Ji, S. H. Jeon, and J. T. Seo, “Anomaly Detection Method Considering PLC Control Logic Structure for ICS Cyber Threat Detection,” *Applied Sciences*, vol. 15, no. 7, pp. 3507, Apr. 2025. doi: 10.3390/app15073507
- [3] J. Mem, K. Hatch, R. Silva, C. Hickert, T. Sookoor, and M. J. Kochenderfer, “Autonomous Attack Mitigation for Industrial Control Systems,” in *Proc. 2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, Baltimore, MD, USA, Jun. 2022, pp. –, doi: 10.1109/DSN-W54100.2022.00015.
- [4] Chen et al., 2024 – “Interpretable Cross-Layer Intrusion Response System Based on Deep Reinforcement Learning for Industrial Control Systems”. *IEEE Transactions on Industrial Informatics*
- [5] S. Sun et al., “A Reinforcement Learning Engine with Reduced Action and State Space for Cyber-Physical Optimal Response,” arXiv preprint arXiv:2410.04518, Oct. 2024. [Online]. Available: <https://arxiv.org/abs/2410.04518>
- [6] K. S. Radoglou-Grammatikis, P. Sarigiannidis, and A. Mavropoulos, “Defending Industrial Internet of Things Against Modbus/TCP Threats: A Combined AI-Based Detection and SDN-Based Mitigation Solution,” *International Journal of Information Security*, vol. 24, 2025. doi: 10.1007/s10207-025-01076-2.
- [7] S. Alqithami, “Hierarchical Adversarially-Resilient Multi-Agent Reinforcement Learning for Cyber-Physical Systems Security,” arXiv preprint arXiv:2506.22445, Jun. 2025. [Online]. Available: <https://arxiv.org/abs/2506.22445>
- [10] "Intrusion Detection and Mitigation Method for the Industrial Internet of Things Using Bidirectional Convolutional Long Short-Term Memory and Deep Recurrent Convolutional Q-Networks
- [11] CONTRASTIVE-KAN: A SEMI-SUPERVISED INTRUSION DETECTION FRAMEWORK FOR CYBERSECURITY WITH SCARCE LABELED DATA
- [12] Network Intrusion Response using Deep Reinforcement Learning in an Aircraft IT-OT Scenario
- [8] Jenifer Paulraj et al., “Autonomous AI-based Cybersecurity Framework for Critical Infrastructure: Real-Time Threat Mitigation” *Processes*, vol. 13, no. 4, p. 1144, Apr. 2025. [Online]. Available: <https://arxiv.org/abs/2507.07416>.
- [9] Toward an Automated Learning Control Architecture for Cyber-Physical Manufacturing Systems

Schedule

	7	8	9	10	11	12	1	2	3	4	5	6
Literature Survey												
Proposal Writing												
Environment												
Implementation												
First Complete Result												
Experiment												
Thesis Writing												

Appendix: Automated Response

RL Response Performance across Algorithms



Observation

- DQN/DDQN achieve high and stable performance (~85–95% success, reward ~5–7), while PPO remains significantly lower (~45–70%, negative/low reward).
- DQN and DDQN show similar convergence, PPO fluctuates and fails to stabilize.

Why

- The environment has discrete actions and simple reward structure, favoring value-based methods (DQN/DDQN).
- PPO requires smoother rewards and more samples, leading to unstable learning in this setting.